



Date: Oct 2, 2025

WEBSITE HANDLING



(Playwright + TypeScript)

WebTables (also called **HTML Tables**) are one of the most common elements in web applications. They display data in rows and columns, and we need to interact with them frequently in automation!

! IMPORTANT!

WebTables consist of `<table>`, `<tr>` (rows), `<th>` (headers), and `<td>` (cells). Understanding this structure is KEY to handling them!

Understanding Table Structure



Sample Table Structure:

☒ Select	Employee Name	Department	Salary
☐	John.Smith	IT	\$75,000
☐	Joe.Root	HR	\$65,000



HTML Structure:

```
<table>
```

```
<tr> <!-- Row -->
```

```
<td>checkbox</td> <!-- Cell -->
```

```
<td>John.Smith</td>
```

```
<td>IT</td>
```

```
</tr>  
</table>
```

Common WebTable Tasks

- ✓ Select a specific user's checkbox
- ✓ Get data from a specific row
- ✓ Find data based on username/condition
- ✓ Get all column values
- ✓ Navigate between siblings (previous/next columns)

Task 1: Select User Checkbox Using XPath

We want to click the checkbox for a specific user (like "Joe.Root"). We'll use **preceding-sibling** axis!

Visual Flow:

- 1 Find the td with text "Joe.Root"
- 2 Go to preceding-sibling td (the one BEFORE it)
- 3 Find the checkbox input inside that td
- 4 Click it! 

XPath Approach

```
// XPath: Find user and go to preceding sibling  
await page.locator(`//td[text()='Joe.Root']/preceding-sibling::td/input`)  
  
await page.locator(`//td[text()='John.Smith']/preceding-sibling::td/input`)
```

XPATH AXIS EXPLAINED!

preceding-sibling:: goes to the previous td in the same row (the checkbox column!)

Task 2: Select User Using CSS

CSS approach is cleaner! We use `:has()` pseudo-class to find the row containing specific text.



CSS Approach

```
// CSS: Find the row that has the username, then get first td
await page.locator(`tr:has(td:text('John.Smith'))`)
  .locator('td')
  .first()
  .click();
```



How it works:

1. `tr:has(td:text('John.Smith'))` → finds the entire row
2. `locator('td')` → gets all td in that row
3. `first()` → selects the first td (checkbox column!)

Task 3: Get All User Data from Row

Want to get ALL data for a specific user (Department, Salary, etc)? Use **following-sibling** !



Get User Data

```
// Get all column values after the username
let userData: string[] = await page
  .locator(`//td[text()='John.Smith']/following-sibling::td`)
  .allInnerTexts();

console.log(userData);
// Output: ['IT', '$75,000']
```



What happens:

following-sibling::td finds ALL td elements that come AFTER the username td in the same row! Perfect for getting all remaining data! 🎯

Creating Reusable Functions

Let's make our code **reusable** and **maintainable**!

Function 1: Select User (XPath)

Reusable Function

```
async function selectUser(  
  page: Page,  
  username: string  
) : Promise<void> {  
  await page.locator(  
    `//td[text()='${username}']/preceding-sibling::td/input[@type=  
  ]).click();  
}
```

// Usage:

```
await selectUser(page, 'Joe.Root');  
await selectUser(page, 'John.Smith');
```

Function 2: Select User (CSS)

Reusable Function

```
async function selectUserWithCss(  
  page: Page,  
  username: string  
) : Promise<void> {  
  await page.locator(`tr:has(td:text('${username}'))`)  
    .locator('td')  
    .first()  
    .click();  
}
```

// Usage:

```
await selectUserWithCss(page, 'Joe.Root');
```

Function 3: Get User Data

Reusable Function with JSDoc

```
/**
 * This method returns the user data from other columns
 * @param page - Playwright Page object
 * @param username - Employee username
 * @returns Array of strings with user data
 */
async function getUserData(
  page: Page,
  username: string
): Promise<string[]> {
  return await page.locator(
    `//td[text()='${username}']/following-sibling::td`
  ).allInnerTexts();
}

// Usage:
let johnData = await getUserData(page, 'John.Smith');
console.log(johnData); // ['IT', '$75,000']
```

XPath Axes Cheatsheet

Axis	What it does	Example Use
preceding-sibling::	Goes BACKWARD (left)	Find checkbox
following-sibling::	Goes FORWARD (right)	Get all data columns
ancestor::	Goes UP (parent/grandparent)	Find parent row



REMEMBER!

- XPath axes are SUPER powerful for navigating tables!
- CSS :has() is cleaner for modern browsers
- Always create reusable functions for common operations

X COMMON MISTAKES

- × Using wrong axis (preceding vs following)
- × Forgetting to handle dynamic table data
- × Not waiting for table to load before interacting
- × Hardcoding row numbers (use text-based locators!)
- × Not handling pagination in large tables

Real World Example 🌐

🇮🇹 Employee Management System:

```
// Complete Example
await page.goto('https://naveenautomationlabs.com/opencart/ui/we

// Select multiple users
await selectUser(page, 'Joe.Root');
await selectUser(page, 'John.Smith');

// Get user details
let johnData = await getUserData(page, 'John.Smith');
console.log('John Data:', johnData);
// Output: ['IT', '$75,000']

let joeData = await getUserData(page, 'Joe.Root');
console.log('Joe Data:', joeData);
// Output: ['HR', '$65,000']
```

🌟 PRO TIPS!

- 💡 Use text-based locators instead of index (more stable!)
- 💡 Create utility functions for common table operations
- 💡 Combine CSS and XPath based on readability
- 💡 Add JSDoc comments to your functions for better documentation
- 💡 For complex tables, consider using ancestor:: axis

✨ KEY TAKEAWAYS

- Use `preceding-sibling::` to go left (find checkbox)
- Use `following-sibling::` to go right (get data columns)
- CSS `:has()` + `text()` is cleaner for finding rows
- Create reusable functions for maintainable tests
- XPath axes are your best friend for table navigation! 🎯

NAVEEN AUTOMATION LABS

Your Journey to Test Automation Excellence 🎯